

28 Sep 2026

PYTHON

Basic Python :

index()

replace() : `string.replace(old, new, count)`

Use code with caution.

- **old**: The text or character you want to search for and remove.
- **new**: The text or character you want to insert in its place.
- **count** (*Optional*): The maximum number of replacements to make, starting from the left. If you leave this out, it replaces every single

replace() is case-sensitive! It won't match capital "Apples"

```
clean_text = text.replace("final", "draft", 1)
```

```
print(clean_text)
```

```
# Output: "user_name_draft_v2_final.txt"
```

Escape characters

-> Working with numbers :

- **str()** : will convert the number directly into the string
- **abs(-5)** : output -> 5
- **pow(4,2)** : 16
- **max(2,3)** : 3
- **min(-1, 8)** : -1
- **round(3.2)** : 3 and **round(3.7)** : 4
- **From math import *** :
 - **sqrt()**
 - **floor()**
 - **ceil()**

- **math.pi**: The mathematical constant
- **math.e**: The mathematical constant
- **math.inf**: A floating-point positive infinity (use **-math.inf** for negative infinity).
- **math.nan**: A floating-point "Not a Number"

Lists []

- **extend(iterable)** : to add another list items into it , result in single flattened list
- **append(item)** : to add one item as a single element , result in nested list
- **insert(index_position, value)** : ex insert(1, "Kevin")
- **remove(value)** : delete specific item from the list
- **clear()** : deletes every items inside the list
- **pop()** : to remove top most value inside the value
- **index(value)** : ex - alphabets.index["a"] => 1
- **sort()** : to sort the values of list
- **reverse()**
- **copy()**
- **count()**

Touples ()

- We can't change tuples values this is IMMUTABLE like a string in java
- It will follow the insertion order
- **.count()** and **.index()**

Dictionaries { }

- Directly accessing the value so : dicVal [1]
- Or another way is : dicVal.get(1) <- here in get we'll pass key
 - If i dont get a key which is not present in dictionary we can pass **default value** to the argument

- Example : `dictVal.get(1, "Not present in the dictionary")`
- `car["year"] = 2024` # Adds a new key-value pair
- `car["model"] = "GT"` # Overwrites the existing value
- `print(car)` # Output: `{'brand': 'Ford', 'model': 'GT', 'year': 2024}`
- `del dict[key]`: Deletes the specified key. Raises an error if missing.
- `.pop(key)`: Deletes the key and **returns** its value.
- `.clear()`: Wipes the entire dictionary clean.
- `.values( )` : to get only the values from the dictionary
- `.items( )` : to get key and value from the dictionary
- `.keys( )` : to get keys only from the dictionary
- `.update( )` : to merge another dictionary into the current one.

Python Collection Hierarchy

Java Interface / Class	Python Abstract Class (<code>collections.abc</code>)	Python Concrete Implementation
<code>Iterable</code>	<code>Iterable</code>	Any object implementing <code>__iter__</code>
<code>Collection</code>	<code>Collection</code>	Built-in collections (<code>list</code> , <code>set</code> , <code>dict</code>)
<code>List / ArrayList</code>	<code>Sequence / MutableSequence</code>	<code>list</code>
<code>LinkedList</code>	<code>MutableSequence</code>	<code>collections.deque</code>

Set / HashSet	Set / MutableSet	set
SortedSet / TreeSet	No direct built-in	Third-party sortedcontainers.SortedSet
Map / HashMap	Mapping / MutableMapping	dict
Queue / ArrayDeque	Varies by use-case	collections.deque or queue.Queue

Set { }

- **set ()** : to create an empty set
- **. add ()** : to add an element to the set
 - **colors.add("blue")** # Adds a single element
- **. update ()** :
 - **colors.update(["yellow", "orange"])** # Adds multiple elements from a list
- **.remove ("red")** : Removes "red". Crashes with KeyError if "red" doesn't exist.
- **.discard("purple")** : Removes "purple". Does NOT crash if it doesn't exist.
- **.pop ()** : Removes and returns an arbitrary item (since sets are unordered)

In mathematical ways :

Operation	Method Syntax	Operator Syntax	What it does	Result
Union	<code>set_a.union(set_b)</code>	<code>set_a set_b</code>	Combines all unique	{1, 2, 3, 4, 5, 6}

			elements from both sets.	
Intersection	<code>set_a.intersection(set_b)</code>	<code>set_a & set_b</code>	Returns only items present in <i>both</i> sets.	<code>{3, 4}</code>
Difference	<code>set_a.difference(set_b)</code>	<code>set_a - set_b</code>	Returns items in <code>set_a</code> that are <i>not</i> in <code>set_b</code> .	<code>{1, 2}</code>
Symmetric Diff.	<code>set_a.symmetric_difference(set_b)</code>	<code>set_a ^ set_b</code>	Returns items in either set, but <i>not</i> both.	<code>{1, 2, 5, 6}</code>

Functions ()

- Parameters and arguments
- Default and keyword function arguments
- `*args` and `**kwargs` :
 - **`*args`** (Arguments): Gathers extra positional arguments into a **tuple**.
 - **`**kwargs`** (Keyword Arguments): Gathers extra named/keyword arguments into a **dictionary**.
 - *The words `args` and `kwargs` are just standard conventions. Only the asterisks (`*` and `**`) are required.*

- **Function scope :**

Function scope (often called local scope) determines where variables can be seen or accessed within your code. In Python, any variable created *inside* a function is locked inside that function; it cannot be read or modified from the outside.

Python manages variable scopes using the LEGB rule, which stands for Local, Enclosing, Global, and Built-in.

1. Local Scope (Inside the Function)

Variables created inside a function are completely private to that function. They are created when the function runs and are destroyed as soon as the function finishes.

- `def my_function():`
- `local_var = "I am secret"`
- `print(local_var)` *# Works perfectly!*
- `my_function()`
- `print(local_var)`

2. Global Scope (Outside all Functions)

A variable created in the main body of your Python script is global. Any function in your file can read it.

- `global_var = "I am everywhere"`
- `def read_global():`
- `print(global_var)` *# Functions can freely read global variables*
- `read_global()` *# Output: I am everywhere*

The Trap: Modifying Global Variables

While functions can read global variables, they cannot modify them by default. If you try to change a global variable inside a function, Python will accidentally create a *new, local variable* with the same name instead.

The Mistake:

- `score = 10`
- `def update_score():`
- `score = 20` *# Python thinks this is a NEW local variable!*
- `update_score()`
- `print(score)` *# Output: 10 (The global variable didn't change!)*

The Fix: The `global` Keyword

To tell a function that you want to modify the actual global variable, use the `global` keyword.

- `score = 10`
- `def update_score():`
- `global score` *# Links this function to the global variable*

- `score = 20`
- `update_score()`
- `print(score)` # *Output: 20 (Successfully updated!)*

3. Enclosing Scope (Nested Functions)

When you put a function inside another function, the inner function can read variables from the outer function. This is called the enclosing scope.

If you want the inner function to *modify* a variable in the outer function, use the `nonlocal` keyword.

- `def outer_function():`
- `outer_var = "Hello"`
- `def inner_function():`
- `nonlocal outer_var` # *Targets the variable in the outer function*
- `outer_var = "World"`
- `inner_function()`
- `print(outer_var)` # *Output: World*
- `outer_function()`

Lambda functions :

- `lambda arguments: expression`

A **lambda function** in Python is a small, anonymous function that is defined **without a name** using the `lambda` keyword.

Unlike a standard function defined with `def`, a lambda function is a **single-line expression**. It evaluates and automatically returns its result without requiring an explicit `return` statement.

Basic Syntax

`lambda arguments: expression`

- **arguments**: A comma-separated list of inputs (just like regular function parameters).
 - **expression**: A single piece of code that operates on the arguments and is returned instantly.
-

Basic Examples

1. A Simple Addition Lambda

```
# Standard Function
def add(x, y):
    return x + y

# Equivalent Lambda Function
add_lambda = lambda x, y: x + y
print(add_lambda(5, 3)) # Output: 8
```

2. Using Conditional Logic (If-Else)

You can include simple conditional logic inside a lambda expression using Python's ternary operator syntax.

```
check_even = lambda num: "Even" if num % 2 == 0 else "Odd"
print(check_even(7)) # Output: Odd
print(check_even(4)) # Output: Even
```

When to Actually Use Lambda Functions

You rarely assign a lambda function to a variable name like the examples above (it's better practice to use `def` for that). Instead, lambdas excel when you need a **throwaway function for a brief moment**, typically passed as an argument to high-order functions like `sort()`, `map()`, or `filter()`.

1. Sorting Complex Data (The `key` parameter)

This is the most common real-world use case for lambda functions.

```
# A list of tuples representing (product, price)
products = [("Laptop", 1200), ("Mouse", 25), ("Monitor", 300)]
# Sort the list by the price (index 1 of each tuple)
products.sort(key=lambda item: item[1])
print(products)
# Output: [('Mouse', 25), ('Monitor', 300), ('Laptop', 1200)]
```

2. Transforming Data with `map()`

Modifies every item in an iterable.

```

numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # Output: [1, 4, 9, 16]

```

3. Selective Filtering with `filter()`

Extracts elements that meet a true/false condition.

```

ages = [12, 18, 25, 40, 15]
adults = list(filter(lambda age: age >= 18, ages))
print(adults)  # Output: [18, 25, 40]

```

Limitations of Lambda Functions

- **Single Expression Only:** They cannot contain loops, multiple statements, or complex logic.
- **Harder to Debug:** Because they don't have names, traceback errors will simply show them as `<lambda>`, making code harder to troubleshoot.

String “ ”, ‘ ’, “ ”, “ ”, “ ”, “ ”

- String is **immutable** in Python
- It will also have +ve and -ve index
- String pool won't allow you to store duplicate values
 - If the values are same then it will give the same reference to that object
- There is also a concept of **SCP (String Constant Pool) & Heap memory.**
- **Doc Strings :** “ ”, “ ”, “ ”, “ ”

File I/O Handling in python:

Mode	Description
'r'	Read-only. Raises I/O error if file doesn't exist.
'r+'	Read and write. Raises I/O error if the file does not exist.
'w'	Write-only. Overwrites file if it exists, else creates a new one.
'w+'	Read and write. Overwrites file or creates new one.
'a'	Append-only. Adds data to end. Creates file if it doesn't exist.
'a+'	Read and append. Pointer at end. Creates file if it doesn't exist.
'rb'	Read in binary mode. File must exist.
'rb+'	Read and write in binary mode. File must exist.
'wb'	Write in binary. Overwrites or creates new.
'wb+'	Read and write in binary. Overwrites or creates new.
'ab'	Append in binary. Creates file if not exist.
'ab+'	Read and append in binary. Creates file if it does not exist.

- Make sure to close opened file
- Can be used as storing our data inside a file

```
main.py x +
main.py
5 # text = f.read()
6 # print(text)
7 # f.close()
8
9 # WRITING A FILE
10
11 f = open('myfile.txt', 'a')
12 f.write('Hello, world!')
13 f.close()
14
15 with open('myfile.txt', 'a'):
16     f.write("Hey I am inside with")
```

- **readLine()** : use to read one line from the file f
- We can also use our file as the small database
- **writelines()** : to write multiple lines inside the file by accessing one by one line from the list

```
16
17 f = open('myfile2.txt', 'w')
18 lines = ['line 1\n', 'line 2\n', 'line
19         3\n']
20 f.writelines(lines)
21 f.close()
```

- **seek()** : **seek()** moves the file cursor to a specific byte position within an open file, enabling random access rather than strictly sequential reading or
- **tells()** : tells functions return the current position on which we are, **like** if we are seeking a data from the file from its 6th position the pointer will go on 6th position and if we'll run **read()** method it will read the data from 6th position.
- **truncate()** : it used for cutting all the character after that specific byte index ex : **truncate(5)** so it will remove all the characters after 5.

if / elif / else statement

- and, or, not()
- <= , >= , != , == , > , <

VIRTAUL ENVIRONMENT :

```
4  
5 '''  
6 virtual environment --> isolated environment, it specific to the project  
7  
8 if your working with multiple project at same time  
9 each and every project have thir own configuration  
10 |  
11
```

Json In Python :

- We use JSON in Python files because it is the universal standard for exchanging, saving, and configuration data between different systems. Since Python cannot natively understand data formats from other technologies, JSON acts as a universal translator.
- To use this in python we need to **import json** in python file
- It has several methods with that we can deal with **JSON** data, these are as follows :
 - **json.loads(json_variable)**
 - **json.dumps(data, indent=numeric_value, sort_keys=True/false)**
: where indent use to make the structural formatted data of json_variable
- We can also work on **JSON Files** :
 - **While** loading the **json** file make sure a **note** that unlike using loads() in simple py file we will use **load()** while working with json files
 - **dump(data , file_wanna_dump_in)** : in **data** we'll pass the data at that place, whereas in **second parameter** we will pass the file name where we want to write our **python** data inside **json file**

Where we can use JSON and Python Simultaneously :

```
python

import json

broken_json_string = '{"name": "John" "age": 30}' # Missing a comma!

try:
    data = json.loads(broken_json_string)
except json.JSONDecodeError as e:
    print(f"Invalid JSON format detected: {e}")
```

Actually there are loads() and dumps() method for Json to python interaction but there are multiple ways and scenarios of using these methods. Like in above python code it is using for catching the json format data exception

Quick Memory Aid: dump vs load

Function	Target	Action
<code>json.load()</code>	File	Reads a file and turns it into Python data.
<code>json.loads()</code>	String	Reads a string and turns it into Python data. (<i>The 's' stands for string</i>)
<code>json.dump()</code>	File	Takes Python data and saves it into a file .
<code>json.dumps()</code>	String	Takes Python data and converts it into a text string .

CSV (Comma Seperated Values) In Python :

- To use this in python we need to **import csv** in python file
- It has several methods with that we can deal with **JSON** data, these are as follows
 - **csv.reader(csv_file_name)** : to read all the values from the csv file
 - **next(csv_reader_extracted_data_python_variable)** : use to skip the first line from the csv extracted data
 - **csv.writer()** : to open up the writer object for us and allow us to write inside the file
 - **csv.writerow(csv line)** : provides us to write inside the csv file

Modules :

- **Modules** helps us to use the functionality of classes inside our code without writing the same methods and implementation again.
- **Packages** : It is kind of folder which contains multiple modules we can say module is .py files
- You can refer this to check all the **built in or External modules** in python <https://docs.python.org/3/py-modindex.html>
- There are two types of modules in python :
 - **Built In Module** : ex. Sys, os, math, collections, json, csv etc..
 - **External Modules** : It is inside the **External Libraries** folder in our python project and it contains these modules like : bnb, [cgkit.py](#), base64, base32 etc.

- We can also install third party module in python which is not inside the external modules 👍

Note

python-docx versions 0.3.0 and later are not API-compatible

python-docx is hosted on PyPI, so installation is relatively easy.

python-docx may be installed with pip if you have it available.

```
pip install python-docx
```

python-docx can also be installed using easy_install, if available.

```
easy_install python-docx
```

If neither pip nor easy_install is available, it can be installed by unpacking the tarball, and running setup.py:

```
tar xvf python-docx-{version}.tar.gz
cd python-docx-{version}
python setup.py install
```

- **Pip** : we can use **pip** to install python modules : by using
 - **pip install module_name**
 - It refers to the package manager allows us to install, manage, uninstall python modules
 - We can use **pip** on terminal, or on command prompt.
 - Pip is pre-installed in python 3
 - After successfully installation of module the module get placed under the :
 - **External libraries / lib / site-packages / module_name**
 - We can also uninstall the module by using :
 - **pip uninstall module_name**

-